

PocketTrainer

Follow-up Review — Verifying Fixes + Firewall / Encryption / SIEM Deep-Dive

Prepared using: NIST Cybersecurity Framework (CSF) 2.0

REPORT DATE September 14, 2026

BASELINE PocketTrainer_Security_Audit_Report.pdf
(first review, this report's predecessor)

METHOD Source diff + Git history re-analysis of the
newly supplied archive vs. the prior review

CLASSIFICATION CONFIDENTIAL — security-sensitive findings

1

CRITICAL

0

HIGH

6

MEDIUM

2

LOW

Table of Contents

- 1. Executive Summary
- 2. Scope & Methodology
- 3. Remediation Verification (Prior Findings)
- 4. Open Findings Register (This Review)
- 5. Detailed Findings
- 6. Firewall & Network Security
- 7. Encryption (In Transit & At Rest)
- 8. SIEM & Security Monitoring
- 9. Other Attack-Surface Review
- 10. NIST CSF 2.0 Function Alignment
- 11. Prioritized Remediation Roadmap
- 12. Recommendations Quick-Reference
- 13. Positive Observations
- 14. Appendix — Methodology & Files Reviewed

1. Executive Summary

This is a follow-up review of PocketTrainer, comparing a newly supplied copy of the project against the prior audit report and re-examining the Git history of both archives. The goal was twofold: verify which of the 11 previously reported findings were actually fixed, and extend the review into four areas specifically requested for this pass — **firewall/network security, encryption, SIEM/monitoring, and other attack-surface safety** (injection, XSS, CSRF, DoS, SSRF, and supply-chain risk).

The team clearly acted on the previous report. A commit explicitly titled "feat: implement security audit fixes (rate limiting, secure storage, zod validation, bounded llm input)" is present in the new history, and it shows: rate limiting was added to login/register/password-reset, the mobile app now stores its auth token in platform secure storage (Keychain/Keystore) instead of AsyncStorage, request bodies are now validated with zod (including an 8-character password policy with complexity rules), CORS is restricted to an explicit origin allowlist, the JWT fallback secret was removed, and a centralized production error handler was added. Seven of the eleven original findings are fixed or meaningfully improved — see Section 3 for the full verification.

However, the single most important action was not completed. The team did purge the leaked secrets from Git history (the commit that originally added backend/.env no longer contains that file anywhere in history), but the **actual credential values were never rotated** — the MongoDB password, JWT signing secret, Gmail app password, and RapidAPI key in the newly supplied backend/.env are byte-for-byte identical to the ones already flagged as compromised in the prior report. Rewriting Git history does not invalidate a secret that was already exposed on a public GitHub remote — **this finding remains CRITICAL and open**, and is the top priority of this report exactly as it was in the last one.

The deep-dive into firewall, encryption, SIEM, and broader attack-surface topics found a reasonably solid application-layer baseline (bearer-token auth with no CSRF exposure, Mongoose + zod largely closing off NoSQL injection, TLS-by-default via Vercel and MongoDB Atlas) but essentially **no infrastructure-layer hardening or observability**: no evidence of a WAF/firewall configuration, no centralized logging or SIEM integration, and no alerting on authentication anomalies. These are detailed in Sections 6–9.

2. Scope & Methodology

In scope:

- Everything in scope for the original review (mobile client, Express/MongoDB backend, third-party integrations, secrets handling) — see the prior report for the full list.
- New/changed files since the last review: backend/middleware/rateLimiter.js, services/tokenService.js, context/AuthContext.jsx, and the updated server.js, authController.js, chatController.js, workoutController.js.
- A dedicated deep-dive into firewall/network posture, encryption in transit and at rest, SIEM/monitoring capability, and a broader attack-surface pass (injection, XSS, CSRF, DoS, SSRF, supply-chain).

Method: the two archives (original and newly supplied) were extracted independently, source files were diffed to identify what changed, and both copies' Git histories were replayed (`git log --all`, `git show`) to confirm whether specific commits — the secret's original addition, its deletion, and the fix commit — behaved as claimed. As before, this is a static/manual source review; no dynamic testing was performed against the live Vercel deployment, MongoDB Atlas cluster, or GitHub repository, and no infrastructure-level settings (Atlas Network Access rules, Vercel Firewall configuration) were directly inspected since they are not stored in source control.

Out of scope (unchanged from prior report): mobile OS-level hardening (root/jailbreak detection), the internals of the third-party Modal AI model, and any settings that live only in the Vercel or MongoDB Atlas dashboards.

3. Remediation Verification (Prior Findings)

Status of every finding from the previous report, based on a direct diff of the source and a replay of both archives' Git history.

#	Prior finding	Status	Evidence
F1	Live secrets committed to Git history / plaintext .env	PARTIALLY FIXED	History was purged (the commit that added backend/.env no longer contains it in either archive's history), but the Mongo password, JWT secret, Gmail app password, and RapidAPI key are unchanged — never rotated. See F12.
F2	No rate limiting/lockout on auth endpoints	FIXED	New backend/middleware/rateLimiter.js (authLimiter, otpLimiter, chatLimiter) is wired into /login, /register, /forgot-password, /reset-password. chatLimiter itself is unused — see F14.
F3	Weak JWT secret fallback + long-lived non-revocable tokens	PARTIALLY FIXED	generateToken() now throws if JWT_SECRET is unset (fallback removed) and expiry was shortened 30d → 7d. No server-side revocation/refresh mechanism was added — see F13.
F4	Overly permissive CORS (all origins allowed)	FIXED	server.js now uses an explicit allowedOrigins allowlist with a callback-based origin check.
F5	Auth token stored unencrypted in AsyncStorage	FIXED	New services/tokenService.js uses expo-secure-store (Keychain/Keystore) on native platforms, with AsyncStorage retained only as a reasonable web-platform fallback.
F6	No password strength enforcement	FIXED	authController.js now defines a zod passwordSchema (8+ chars, 1 uppercase, 1 digit) applied to register, reset, and update-password flows.
F7	Unvalidated/unbounded input forwarded into AI prompts	FIXED	chatController.js now enforces a 500-character MAX_MESSAGE_LENGTH before any AI call, and getChatHistory() is now bounded/paginated (limit, capped at 100) — this also resolves the earlier 'no chat pagination' recommendation.
F8	Raw internal error messages returned to clients	PARTIALLY FIXED	server.js gained a centralized production error handler, and authController.js/chatController.js correctly call next(error) to use it. workoutController.js was not updated and still returns res.status(500).json({message: error.message}) directly on all four of its routes — see F15.
F9	No structured logging, monitoring, or alerting	NOT FIXED	Still only console.log/console.error; no log aggregation, APM, or alerting was found — see F17.
F10	No documented incident-response / secret-rotation runbook	NOT FIXED	No runbook file was found. The team did act on this report once (a real fix commit exists), which is a good sign, but there is still nothing written down — see F18.
F11	Full Git history, node_modules, and logs shipped in archive	NOT FIXED	The new archive still includes the full .git directory and node_modules for both app and backend — see F21.

Scorecard: 6 fixed, 3 partially fixed, 2 not fixed, out of 11. That's genuine, verifiable progress — but the two "not fixed" items (SIEM/monitoring, incident response) and the "partially fixed" F1 are exactly the ones this report's requested focus areas (firewall, encryption, SIEM, broader attack safety) bear on most directly.

4. Open Findings Register (This Review)

Ten items require action: one carried-forward critical finding, six medium findings (three carried forward, three newly identified in this pass), two new low-severity items, and one informational hygiene note.

#	Finding	Severity	Primary NIST CSF 2.0 Mapping
F12	Leaked credentials still unrotated (history purged, values unchanged)	CRITICAL	PR.AA-05, GV.SC-05
F13	JWT tokens still cannot be revoked server-side (7-day non-blacklisted lifetime)	MEDIUM	PR.AA-01, PR.AA-05
F14	chatLimiter implemented but never applied to the chat/AI route	MEDIUM	PR.AA-03, ID.RA-01
F15	workoutController.js bypasses the new centralized error handler	MEDIUM	PR.PS-01
F16	Schema/persistence mismatch silently defeats the OTP resend cooldown	MEDIUM	PR.AA-03, PR.DS-01
F17	No centralized logging, SIEM integration, or alerting	MEDIUM	DE.CM-01, DE.AE-02
F18	No documented incident-response / secret-rotation runbook	MEDIUM	RS.MA-01, RC.RP-01
F19	CORS sets credentials:true despite bearer-token (non-cookie) auth	LOW	PR.PS-05
F20	Unused 'resend' dependency adds supply-chain surface	LOW	GV.SC-05
F21	Full Git history, node_modules, and prior audits shipped in archive	INFO	GV.SC-05, ID.AM-01

5. Detailed Findings

F12. Leaked credentials still unrotated (history purged, values unchanged)

CRITICAL

NIST CSF 2.0 mapping: PROTECT — PR.AA-05 (credentials and secrets managed); GOVERN — GV.SC-05 (supply-chain / secrets policy)

Impact: The newly supplied backend/.env contains the exact same MongoDB Atlas password, JWT signing secret, Gmail app password, and RapidAPI key as the file flagged CRITICAL in the previous report. On the positive side, the Git history in this archive confirms the secret was purged: the commit that originally added backend/.env (previously hash 018cd64, now re-written as a different hash after history rewrite) no longer contains that file in its tree, and no commit anywhere in history adds backend/.env. That's real, correct remediation of the history-hygiene half of the original finding. But purging history only stops *new* clones from finding the secret going forward — it does nothing to un-expose a secret that was already pushed to a remote (github.com/umarsoomro1/pocket-trainer.git) before the rewrite. Anyone who cloned, forked, or viewed the repository in that window still has these exact working credentials today.

Evidence:

```
backend/.env (new archive) → same MONGO_URI, JWT_SECRET, EMAIL_PASS, RAPIDAPI_KEY, AI_API_KEY  
values as the previous report's redacted findings
```

```
git show <rewritten-commit>:backend/.env → "fatal: path 'backend/.env' exists on disk, but not in  
'<commit>' " — confirms the file was scrubbed from that commit's tree
```

```
git log --all --diff-filter=A --name-only | grep backend/.env → no results across the full  
rewritten history — the purge was thorough
```

Recommendations:

- This is unchanged from the last report: rotate the MongoDB Atlas database password, JWT_SECRET, Gmail app password, AI/Modal API key, and RapidAPI key now — a history rewrite is not a substitute for rotation.
- After rotating, treat the old values as permanently burned: do not reuse any part of them (e.g. a password variant) in the new secrets.
- Audit MongoDB Atlas access logs and the AI/RapidAPI account's billing/usage history for the window the secrets were exposed, to check for signs of misuse.
- Once rotated, this finding — and only this finding — can be closed; everything else about the F1 remediation was done correctly.

F13. JWT tokens still cannot be revoked server-side

MEDIUM

NIST CSF 2.0 mapping: PROTECT — PR.AA-01 (identities/credentials managed), PR.AA-05 (access permissions managed)

Impact: generateToken() now fails fast if JWT_SECRET is missing (good) and issues 7-day tokens instead of 30-day ones (an improvement). However there is still no refresh-token flow, no token/jti blacklist, and no per-user token-version claim. A token issued before a password change (updatePassword) or a suspected compromise remains valid for up to 7 days with no way to invalidate it early.

Evidence:

```
backend/controllers/authController.js — generateToken() → jwt.sign({ id }, process.env.JWT_SECRET,  
{ expiresIn: '7d' }); no revocation list or tokenVersion check anywhere in authMiddleware.js
```

Recommendations:

- Add a per-user tokenVersion (or similar) field, incremented on password change/logout-everywhere, and checked in authMiddleware.protect() alongside signature verification.
- Consider a shorter-lived access token (e.g. 15–60 minutes) with a refresh token for longer sessions, so a leaked access token has a much smaller window of usefulness.

F14. chatLimiter implemented but never applied to the chat/AI route

MEDIUM

NIST CSF 2.0 mapping: PROTECT — PR.AA-03 (users/devices authenticated commensurate with risk); IDENTIFY — ID.RA-01 (risk assessment of third-party/AI dependency)

Impact: backend/middleware/rateLimiter.js defines a chatLimiter (20 requests/minute) with a comment specifically noting it "guards against Modal compute credit drainage." It is exported from the module but chatRoutes.js never imports or applies it — only authLimiter and otpLimiter made it into authRoutes.js. As a result the single most compute-expensive, externally-billed endpoint (the one that calls the Modal AI model on every request) has no rate limiting at all, despite the fix already being written and ready to use.

Evidence:

```
backend/middleware/rateLimiter.js – module.exports = { authLimiter, otpLimiter, chatLimiter };
```

```
backend/routes/chatRoutes.js – imports only { protect } from authMiddleware; chatLimiter is never imported or referenced
```

Recommendations:

- Apply the existing chatLimiter to both routes in chatRoutes.js (GET / and POST /) — this is a one-line wiring fix, not new code.
- Since this endpoint calls a billed third-party AI model, also consider a per-user daily/monthly quota in addition to the per-minute limiter, to bound worst-case cost exposure.

F15. workoutController.js bypasses the new centralized error handler

MEDIUM

NIST CSF 2.0 mapping: PROTECT — PR.PS-01 (secure development / secure defaults)

Impact: server.js now has a centralized error-handling middleware that hides internal error detail behind a generic message in production, and authController.js/chatController.js were updated to route failures through it via next(error). workoutController.js was missed: all four of its handlers (generateAndAssignPlan, getToday'sWorkout, getDashboardData, completeWorkout) still catch errors locally and respond with res.status(500).json({ message: error.message }) directly, re-introducing the exact information-disclosure issue the centralized handler was built to close — just on a smaller set of routes.

Evidence:

```
backend/controllers/workoutController.js – 4 occurrences of `return res.status(500).json({ message: error.message ... });` instead of `next(error);`
```

```
Compare with backend/controllers/authController.js / chatController.js, which correctly call next(error) in every catch block.
```

Recommendations:

- Change all four catch blocks in workoutController.js to call next(error) instead of building their own response, so they benefit from the same production-safe error handler as the rest of the API.
- Consider a lint rule or code-review checklist item requiring next(error) in Express catch blocks, so this doesn't regress again as new controllers are added.

F16. Schema/persistence mismatch silently defeats the OTP resend cooldown

MEDIUM

NIST CSF 2.0 mapping: PROTECT — PR.AA-03 (authentication commensurate with risk), PR.DS-01 (data-at-rest integrity)

Impact: As part of the F2 fix, forgotPassword() now checks user.lastOtpSentAt to enforce a 60-second cooldown between reset-code requests — a good defense-in-depth addition on top of otpLimiter. However, the User Mongoose schema (backend/models/User.js) does not declare a lastOtpSentAt field. Mongoose schemas are strict by default: setting a path that isn't declared in the schema is silently dropped on save() and is never persisted to MongoDB. That means user.lastOtpSentAt is always undefined on the next request, the 60-second-cooldown check always evaluates as "never sent before," and the cooldown never actually engages — it only appears to work in code review, not at runtime. The same schema gap affects user.weightHistory (referenced in registerUser and updateUserProfile and read in getDashboardData): pushes to it are silently dropped, so the dashboard's weight-trend chart can never accumulate more than the single most recent data point. That second half is a functional/data-integrity bug rather than a security one, but it stems from the same root cause.

Evidence:

```
backend/models/User.js – schema declares email, password, weight, dob, goal, active_program_id,
current_day_index, last_workout_date, resetPasswordToken, resetPasswordExpires only.
```

```
backend/controllers/authController.js – forgotPassword() reads/writes user.lastOtpSentAt, a path
absent from the schema above.
```

```
backend/controllers/authController.js / workoutController.js – both read/write user.weightHistory,
also absent from the schema.
```

Recommendations:

- Add lastOtpSentAt: { type: Date } and a weightHistory: [{ weight: Number, date: Date }] array to the User schema so these values are actually persisted.
- As a general practice, when adding a new field reference in a controller, grep the corresponding model file first to confirm the field is declared — this class of bug is easy to introduce silently under Mongoose's default strict mode.
- Add a lightweight integration test that registers a user, requests two reset codes in immediate succession, and asserts the second is rejected with 429 — this would have caught the cooldown not actually engaging.

F17. No centralized logging, SIEM integration, or alerting

MEDIUM

NIST CSF 2.0 mapping: DETECT — DE.CM-01 (networks/systems monitored), DE.AE-02 (adverse events analyzed)

Impact: Unchanged from the prior review: observability is still limited to scattered console.log/console.error calls inside individual controllers (e.g. [GENERATION START]/[GENERATION SUCCESS] in workoutController.js, [AI WORKOUT SYNC] in chatController.js, [SERVER ERROR] in the new error handler). On Vercel these lines land in ephemeral per-invocation function logs with no aggregation, no retention policy, no correlation across requests, and no alerting. There is still no way to detect — let alone get notified about — a credential-stuffing wave against /login, a spike in 429s from the new rate limiters, or unusual database query volume, in anything close to real time. See Section 8 for a full SIEM-focused assessment.

Recommendations:

- See Section 8 ("SIEM & Security Monitoring") for the detailed recommendation set — in short: ship structured logs to a log-management/SIEM platform, and alert on authentication and rate-limit anomalies.

F18. No documented incident-response / secret-rotation runbook

MEDIUM

NIST CSF 2.0 mapping: RESPOND — RS.MA-01 (incident response plan executed); RECOVER — RC.RP-01 (recovery plan executed)

Impact: No incident-response documentation was found in either archive. The good news is this round shows the team does have an informal process: the prior report's findings map almost one-to-one onto a real fix commit ("feat: implement security audit fixes..."), which is meaningfully better than the last review's evidence (a report that existed but led to no rotation). The remaining gap is that this process is undocumented and, per F12, incomplete — credential rotation specifically was still missed even though seven other items were handled correctly. A short written runbook would help ensure secret rotation isn't the one step that keeps getting missed.

Recommendations:

- Write a one-page runbook: who to notify, how to rotate each class of secret (DB, JWT, email, third-party API keys), how to force-invalidate sessions, and where to check for signs of misuse.
- Add "rotate any secret that was ever committed, even if since removed" as an explicit, unmissable line item — this is precisely the step that was skipped both times.

F19. CORS sets credentials:true despite bearer-token (non-cookie) auth

LOW

NIST CSF 2.0 mapping: PROTECT — PR.PS-05 (secure configuration baselines)

Impact: The new CORS configuration correctly restricts origins to an allowlist (fixing F4), but still sets credentials:true. That flag exists to allow cookies/HTTP auth to be sent cross-origin; PocketTrainer authenticates exclusively via a Bearer token in the Authorization header, which doesn't need it. It's not exploitable on its own given the origin allowlist is correctly enforced, but it's an unnecessary broadening of the CORS configuration for a capability the app doesn't use, and a common source of confusion if the allowlist logic is ever loosened later.

Evidence:

```
backend/server.js - cors({ origin: (origin, callback) => {...}, credentials: true })
```

Recommendations:

- Remove credentials: true unless/until the app actually adopts cookie-based auth, to keep the CORS policy minimal and matched to what's actually in use.

F20. Unused 'resend' dependency adds supply-chain surface

LOW

NIST CSF 2.0 mapping: GOVERN — GV.SC-05 (supply-chain risk management)

Impact: backend/package.json lists resend (^6.26.0) as a dependency alongside nodemailer, but backend/services/emailService.js only imports and uses nodemailer — resend does not appear to be referenced anywhere in the backend source. Every dependency is additional code (and its own transitive dependencies) that ships to production and needs to stay patched, whether or not it's actually used.

Evidence:

```
grep -rn "resend" backend/services/*.js backend/controllers/*.js → no matches; only  
backend/package.json declares it
```

Recommendations:

- Remove the resend package if it isn't planned for near-term use, or finish wiring it in and remove nodemailer if it's meant to be a replacement — avoid keeping both indefinitely.

F21. Full Git history, node_modules, and prior audits shipped in archive

INFO

NIST CSF 2.0 mapping: GOVERN — GV.SC-05 (supply chain / handling of sensitive artifacts); IDENTIFY — ID.AM-01 (asset inventory)

Impact: As with the previous review, the delivered archive includes the full .git directory (which is precisely how both this review and the last one were able to confirm the F12/F1 secret timeline) plus node_modules and Expo caches. This copy also includes the two previous self-generated audit report files (security_audit_report.pdf/docx) inside the repository itself. None of this is a vulnerability in the running application, but it does mean every time this project is shared as a zip/rar for review or handoff, historical secrets and prior findings travel along with it.

Recommendations:

- When sharing for review or handoff, export a clean copy (git archive from a fresh clone, or exclude .git/, node_modules/, and .expo/) unless full history is specifically needed.

6. Firewall & Network Security

PocketTrainer's backend runs as Vercel serverless functions in front of MongoDB Atlas — there is no self-managed server or on-prem firewall to configure. "Firewall" for this architecture means three separate controls: Vercel's edge-level protections, MongoDB Atlas's network access list, and the application's own request-level throttling. None of the first two are visible from source code, since they're configured in each provider's dashboard rather than in a committed file — so this section states what could and couldn't be verified, and what to check.

Control / Area	Status	Assessment
Vercel Firewall / DDoS mitigation	NEEDS VERIFICATION	Vercel provides basic DDoS protection by default and an optional Firewall/Attack Challenge Mode as a dashboard setting. No <code>vercel.json</code> <code>firewall/rules</code> configuration is present in the repo, so this is either using platform defaults or hasn't been reviewed — confirm in the Vercel project dashboard.
MongoDB Atlas Network Access (IP allowlist)	NEEDS VERIFICATION	Not visible from source. Serverless functions have dynamic egress IPs, which commonly leads teams to allowlist <code>0.0.0.0/0</code> ("allow from anywhere") in Atlas just to keep deployments working — that setting alone would mean the database's network-level firewall is effectively disabled. This should be explicitly checked.
Application-level rate limiting	PARTIALLY FIXED	<code>express-rate-limit</code> is applied to <code>auth/OTP</code> routes (F2, fixed) but not yet to the <code>chat/AI</code> route (F14) — see Section 4/5. This is a useful but not sufficient control on its own: it operates per-IP by default and can be diluted by an attacker using many source IPs.
Payload size limits	FIXED	<code>server.js</code> sets <code>express.json({ limit: '100kb' })</code> , which blocks oversized JSON bodies being used as a crude denial-of-service/resource-exhaustion vector.
Function execution timeout	FIXED	<code>vercel.json</code> caps <code>server.js</code> and <code>api/**/*.js</code> at <code>maxDuration: 60</code> seconds, bounding worst-case compute cost per request (relevant given the paid AI backend call).

Recommendations:

- Confirm and, if not already enabled, turn on Vercel's Firewall / Attack Challenge Mode for the production deployment, and review Vercel's built-in rate-limiting/bot-protection rules as a defense-in-depth layer above `express-rate-limit`.
- Check the MongoDB Atlas Network Access list specifically for a `0.0.0.0/0` entry. If serverless egress IPs make a tight allowlist impractical, use MongoDB Atlas's own IP Access List combined with database-user scoping (least-privilege roles per credential) rather than leaving the network layer fully open.
- Apply the existing `chatLimiter` to the `chat` route (F14) so every externally-billed or database-mutating endpoint has app-layer throttling, not just `auth`.
- Consider fronting the API with a proper WAF (Vercel's own, or Cloudflare in front of the Vercel deployment) if the app's user base or attack exposure grows — application-level rate limiting alone won't stop a distributed attack.

7. Encryption (In Transit & At Rest)

Control / Area	Status	Assessment
Transport encryption — API traffic	FIXED	Vercel serves all deployments over HTTPS/TLS by default and the mobile client's <code>API_BASE_URL</code> in <code>axiosConfig.js</code> is an <code>https://</code> URL. Helmet is applied, providing baseline security headers.
Transport encryption — database connection	FIXED	The <code>MONGO_URI</code> uses the <code>mongodb+srv://</code> scheme, which for MongoDB Atlas negotiates TLS by default for the driver-to-cluster connection.
Encryption at rest — database	NEEDS VERIFICATION	MongoDB Atlas encrypts data at rest by default on its managed clusters, but this is a platform default to confirm in the Atlas project settings rather than something visible from the connection string alone.
Encryption at rest — application secrets	NOT FIXED	<code>backend/.env</code> stores all credentials in plaintext on disk with no encryption or secrets-manager wrapper (this is F12/F1's root cause — see Section 5).
Sensitive data in JWT payload	FIXED (already good)	<code>generateToken()</code> signs only <code>{ id }</code> , no email, password, or health data in the token payload — JWTs are base64-encoded and readable by anyone who intercepts them, so keeping the payload minimal is the right call and was already the case in both reviews.
Field-level encryption for sensitive health fields	NOT FIXED	Date of birth, weight, and weight history are stored as plain fields in MongoDB with no additional field-level encryption. Relies entirely on Atlas's at-rest encryption and standard access control.
Certificate/transport pinning (mobile client)	NOT FIXED	No certificate pinning was found in the Axios configuration — the app trusts the device's standard certificate store, which is normal for most apps but offers less protection against a compromised CA or a MITM proxy on a compromised device.

Recommendations:

- Confirm encryption-at-rest is enabled on the MongoDB Atlas cluster (default on all current Atlas tiers, but worth a one-time confirmation in project settings).
- This is the same underlying problem as F12: move secrets out of a plaintext `.env` and into an encrypted store (Vercel encrypted environment variables at minimum; a dedicated secrets manager for anything beyond a small project).
- For a health/fitness app, consider whether particularly sensitive fields (date of birth) warrant application-level (field) encryption in addition to Atlas's at-rest encryption, as defense-in-depth against a database-level compromise.
- Certificate pinning is optional for most consumer apps and adds maintenance overhead (pin rotation); it's worth considering only if the threat model includes sophisticated on-device attackers, which doesn't appear to be the current priority relative to the other findings in this report.

8. SIEM & Security Monitoring

There is currently no SIEM (Security Information and Event Management) capability of any kind. This section lays out what a minimally viable version would look like for an app at PocketTrainer's stage, since "add a SIEM" isn't itself an actionable step without more specificity.

Control / Area	Status	Assessment
Structured, centralized logging	NOT FIXED	All logging is console.log/console.error scattered across controllers (e.g. [GENERATION START], [AI WORKOUT SYNC], [SERVER ERROR]), with no request IDs, no consistent structure, and no aggregation — Vercel's function logs are short-retention and not designed as a log warehouse.
Authentication anomaly detection	NOT FIXED	No alerting exists for spikes in failed logins, 429 (rate-limited) responses, or repeated OTP failures — exactly the signal that would show F2's rate limiter is under active attack.
Database/query anomaly detection	NOT FIXED	No monitoring on MongoDB Atlas query patterns, connection counts, or unusual data access volume (Atlas itself offers this as a paid feature, but it doesn't appear to be configured/reviewed).
Error/exception tracking (APM)	NOT FIXED	No Sentry/Bugsnag/equivalent integration — errors are only visible by manually reading serverless function logs after the fact.
Audit trail for sensitive actions	NOT FIXED	No audit log for security-relevant events (password changes, failed auth attempts, admin-style actions) separate from ordinary application logs.

Recommendations (roughly in order of effort vs. value):

- **Start with structured logging.** Replace ad hoc console.log calls with a structured logger (pino is a good fit for serverless — low overhead, JSON output) that includes a request ID, user ID (when authenticated), route, and outcome on every request.
- **Ship logs somewhere persistent.** Forward structured logs to a log-management platform with real retention and search — options at this scale include Better Stack (Logtail), Axiom, Datadog, or the ELK/OpenSearch stack if self-hosting is preferred. This is the foundation a SIEM sits on top of.
- **Add an APM/error tracker** (Sentry is the common choice for Node/React Native) so exceptions are captured with stack traces and context automatically, instead of relying on someone noticing a 500 response.
- **Define 3–5 concrete alerts** before reaching for a full SIEM product: a spike in 401/429 responses on auth routes, repeated OTP verification failures for the same account, MongoDB connection errors, and any 500 rate above a small baseline. Route these to email/Slack at minimum.
- **Graduate to a SIEM proper once there's enough log volume/team size to justify it** (e.g. Microsoft Sentinel, Elastic Security, or a lighter SaaS SIEM) — for a project this size, the structured-logging + alerting steps above deliver most of the practical benefit at a fraction of the operational cost.

9. Other Attack-Surface Review

A pass across the classic web/mobile attack categories not already covered as standalone findings above.

Vector	Assessment	Status
NoSQL / query injection	All Mongoose queries observed use validated, typed values (email/password/etc. are parsed through zod schemas as plain strings before reaching <code>User.findOne()</code> , which prevents an attacker from injecting a MongoDB query operator like <code>\$gt</code> via a JSON body). No raw/unsanitized user input was found being interpolated into a query object.	FIXED (already good)
Cross-site scripting (XSS)	The client is a React Native app, not a web view rendering raw HTML — user- and AI-generated text (chat messages, workout notes) is rendered through native Text components rather than an HTML sink, which is the main XSS vector in web apps. Risk is low in the current mobile-only client; revisit if a web/admin dashboard is ever added.	OUT OF SCOPE
Cross-site request forgery (CSRF)	Authentication is entirely Bearer-token-based (Authorization header), not cookie/session-based — CSRF specifically exploits ambient cookie auth, so the current design is not exposed to it. (See F19 for a related, low-severity CORS configuration cleanup.)	FIXED (already good)
Denial of Service (DoS) — request volume	Auth/OTP routes are now rate-limited (F2, fixed); the chat/AI route is not (F14, open) despite the limiter already being written for it.	PARTIALLY FIXED
Denial of Service (DoS) — payload/resource exhaustion	JSON body size capped at 100kb, function execution capped at 60s, and chat messages capped at 500 characters — reasonable bounds against oversized-payload and long-running-request abuse.	FIXED (already good)
Server-Side Request Forgery (SSRF)	Outbound calls from the backend (to the Modal AI endpoint and the RapidAPI ExerciseDB) use fixed, environment-configured URLs/hosts — exercise names are passed as query values, not as URLs or hosts, so user input cannot redirect these requests to an attacker-controlled destination.	FIXED (already good)
Broken Object-Level Authorization (IDOR)	Workout, dashboard, and chat endpoints continue to scope data lookups to <code>req.user</code> (populated from the verified JWT) rather than trusting a client-supplied user/record ID — consistent with the previous review's finding.	FIXED (already good)
Supply-chain / dependency risk	No automated dependency scanning (npm audit / Dependabot / Snyk) is configured in CI, and one unused dependency (resend, F20) is present. <code>package-lock.json</code> is committed, which is good practice for reproducible installs.	NOT FIXED
Secrets in logs	<code>console.log</code> statements reviewed print IDs, plan titles, and status text — no instance of a password, token, or API key being logged was found.	FIXED (already good)

Recommendations:

- Add automated dependency scanning to CI (Dependabot is free and simplest on GitHub; Snyk/npm audit are alternatives) so known-vulnerable packages are flagged automatically rather than relying on manual review.
- Remove the unused resend dependency (F20) to keep the supply-chain surface matched to what's actually used.
- If a web-based admin dashboard or companion web app is ever built, revisit XSS specifically — the "out of scope" status here is a property of the current mobile-only client, not a guarantee that will still hold with a different frontend.

- Apply chatLimiter to close the one remaining DoS gap (F14) — everything else in this section is already in reasonably good shape.

10. NIST CSF 2.0 Function Alignment

Updated from the prior review to reflect verified fixes. Protect has improved the most (rate limiting, CORS, secure storage, input validation all landed); Detect and Respond remain the weakest functions, consistent with F17/F18 and the SIEM assessment in Section 8.

Function	Change since last review	Posture	Key open items
GOVERN (GV)	No change — still no secrets policy or data-classification standard.	Weak	F12, F18, F21
IDENTIFY (ID)	No change — no formal secrets/third-party data-flow inventory.	Partial	F12, F14, F20
PROTECT (PR)	Improved significantly: rate limiting, CORS allowlist, secure token storage, password policy, bounded AI input, and a centralized error handler were all added.	Partial (was Partial/Weak)	F12, F13, F14, F15, F16, F19
DETECT (DE)	No change — still only ad hoc console logging, no anomaly detection.	Weak	F14, F17
RESPOND (RS)	Slightly improved in practice (a real fix commit followed the last report) but still undocumented, and the most important action (rotation) was still missed.	Weak	F12, F18
RECOVER (RC)	No change — no application-level recovery/rotation plan documented.	Not evidenced	F18

11. Prioritized Remediation Roadmap

Immediate

0 – 7 days

- Rotate the MongoDB Atlas password, JWT_SECRET, Gmail app password, and RapidAPI/AI key — still the single highest-priority action (F12).
- Wire the existing chatLimiter into chatRoutes.js — a one-line fix that closes F14.
- Fix workoutController.js's four catch blocks to use next(error) instead of leaking error.message directly (F15).
- Add the missing lastOtpSentAt and weightHistory fields to the User schema so the OTP cooldown actually persists (F16).

Short-term

2 – 4 weeks

- Add a tokenVersion (or equivalent) claim so JWTs can be invalidated on password change/logout (F13).
- Confirm MongoDB Atlas Network Access rules and Vercel Firewall settings; tighten the Atlas IP allowlist if it currently permits 0.0.0.0/0 (Section 6).
- Stand up structured logging (pino) and forward it to a log-management platform, plus add an APM/error tracker like Sentry (Section 8).
- Remove the credentials:true CORS flag (F19) and the unused resend dependency (F20).
- Add automated dependency scanning (Dependabot/Snyk) to CI (Section 9).

Ongoing / Longer-term

1 – 3 months

- Move all secrets to a managed secrets store (Vercel encrypted env vars at minimum, a dedicated secrets manager beyond that) so this class of finding can't recur (F12/Section 7).
- Define 3–5 concrete security alerts (auth failure spikes, repeated OTP failures, 500-rate thresholds) and route them to email/Slack; graduate to a full SIEM only once log volume justifies it (Section 8).
- Write and circulate a one-page incident-response / secret-rotation runbook, explicitly calling out "rotate, don't just remove" (F18).
- Consider a WAF (Vercel Firewall or Cloudflare) in front of the API as the app's user base grows (Section 6).
- Re-run this verification pass after the immediate/short-term items land, the same way this report verified the last one.

12. Recommendations Quick-Reference

A one-line recommendation for every open finding. Full context and evidence for each is in Section 5, 6, 7, 8, or 9 as noted.

#	Recommendation	Severity
F12	Rotate the Mongo, JWT, Gmail, and RapidAPI/AI credentials now — the history purge alone doesn't fix this.	CRITICAL
F13	Add a token-revocation mechanism (tokenVersion claim or refresh tokens) so JWTs can be invalidated early.	MEDIUM
F14	Wire the already-written chatLimiter into chatRoutes.js.	MEDIUM
F15	Change workoutController.js's catch blocks to use next(error) like the rest of the API.	MEDIUM
F16	Add lastOtpSentAt and weightHistory to the User schema so those writes actually persist.	MEDIUM
F17	Add structured logging + a log-management platform + a few concrete security alerts.	MEDIUM
F18	Write a short incident-response / secret-rotation runbook.	MEDIUM
F19	Remove CORS credentials:true since the app doesn't use cookie-based auth.	LOW
F20	Remove the unused resend dependency (or finish integrating it and drop nodemailer).	LOW
F21	Share a clean export (no .git/node_modules) when handing off or reviewing the project.	INFO

13. Positive Observations

Carried forward from the previous review, plus everything fixed since then:

- Rate limiting is now correctly applied to login, register, and OTP request/verification (F2, fixed).
- The mobile client now stores its auth token in platform secure storage (Keychain/Keystore via expo-secure-store) instead of AsyncStorage (F5, fixed).
- Request bodies are now validated with zod, including a real password-complexity policy (F6, fixed) — and this same validation is a big part of why NoSQL injection isn't practically exploitable here (Section 9).
- CORS is now restricted to an explicit origin allowlist instead of allowing all origins (F4, fixed).
- Chat messages are length-capped and chat history is now paginated/bounded, closing both the prompt-injection-adjacent risk and the earlier unbounded-history performance concern in one fix (F7, fixed).
- A centralized, production-safe error handler now exists in server.js, correctly used by the auth and chat controllers (F8, mostly fixed — see F15 for the remaining gap).
- The JWT fallback secret was removed entirely; the app now fails fast if JWT_SECRET is missing (part of F3, fixed).
- Passwords are hashed with bcrypt, Helmet is applied globally, the forgot-password flow avoids account enumeration, and reset codes are hashed before storage — all unchanged and still good practice.
- Authentication is Bearer-token-based rather than cookie-based, which structurally avoids CSRF exposure (Section 9).
- JWT payloads contain only a user ID — no email, password, or health data — limiting what's exposed if a token is intercepted.
- Endpoints continue to scope all data access to the authenticated req.user rather than trusting client-supplied IDs, avoiding IDOR (Section 9).
- Request payload size and function execution time are both capped, and outbound third-party calls use fixed, non-user-controlled URLs, closing off straightforward SSRF and basic resource-exhaustion vectors (Sections 6 and 9).

14. Appendix — Methodology & Files Reviewed

Comparison basis:

- Previous archive: 1788620030803_PocketTrainer.rar, reviewed in PocketTrainer_Security_Audit_Report.pdf (this report's predecessor).
- Current archive: 1789417742385_pocket-trainer.rar, diffed file-by-file against the previous archive's extracted source tree.
- Both archives' .git directories were extracted and replayed independently (git log --all, git show, git ls-tree) to verify claims about history rewriting and the fix commit.

Newly added or renamed files reviewed in this pass:

- backend/middleware/rateLimiter.js (new)
- services/tokenService.js (new)
- context/AuthContext.jsx (renamed from AuthContext.js, logic updated)

Files re-reviewed for changes:

- backend/server.js, backend/controllers/authController.js, chatController.js, workoutController.js
- backend/middleware/authMiddleware.js, backend/routes/authRoutes.js, chatRoutes.js, workoutRoutes.js
- backend/models/User.js, WorkoutPlan.js, ChatMessage.js, DailyLog.js
- backend/services/aiService.js, emailService.js, exerciseService.js
- backend/.env, backend/package.json, backend/vercel.json
- api/axiosConfig.js, screens/RegisterScreen.jsx
- Git history and remote configuration in both archives (analyzed to confirm the F12/F1 secret timeline and the history-purge claim).

This report reflects a manual source-code and Git-history review only. It does not constitute a penetration test, does not verify current MongoDB Atlas network/IAM configuration or Vercel Firewall settings directly (both are called out as "needs verification" in Sections 6–7 rather than assumed), and does not confirm whether the F12 credentials have been misused — that requires reviewing provider-side access logs, which remains outside the scope of a source review.